ROS: an open-source Robot Operating System

Morgan Quigley*, Brian Gerkey†, Ken Conley†, Josh Faust†, Tully Foote†,
Jeremy Leibs‡, Eric Berger†, Rob Wheeler†, Andrew Ng*

*Computer Science Department, Stanford University, Stanford, CA
†Willow Garage, Menlo Park, CA
‡Computer Science Department, University of Southern California

***Abstract*— This paper gives an overview of ROS, an open-source robot operating system. ROS is not an operating system in the traditional sense of process management and scheduling; rather, it provides a structured communications layer above the host operating systems of a heterogenous compute cluster. In this paper, we discuss how ROS relates to existing robot software frameworks, and briefly overview some of the available application software which uses ROS.**

I. INTRODUCTION

Writing software for robots is difficult, particularly as the scale and scope of robotics continues to grow. Different types of robots can have wildly varying hardware, making code reuse nontrivial. On top of this, the sheer size of the required code can be daunting, as it must contain a deep stack starting from driver-level software and continuing up through perception, abstract reasoning, and beyond. Since the required breadth of expertise is well beyond the capabilities of any single researcher, robotics software architectures must also support large-scale software integration efforts.

To meet these challenges, many robotics researchers, including ourselves, have previously created a wide variety of frameworks to manage complexity and facilitate rapid prototyping of software for experiments, resulting in the many robotic software systems currently used in academia and industry [1]. Each of these frameworks was designed for a particular purpose, perhaps in response to perceived weaknesses of other available frameworks, or to place emphasis on aspects which were seen as most important in the design process.

ROS, the framework described in this paper, is also the product of tradeoffs and prioritizations made during its design cycle. We believe its emphasis on large-scale integrative robotics research will be useful in a wide variety of situations as robotic systems grow ever more complex. In this paper, we discuss the design goals of ROS, how our implementation works towards them, and demonstrate how ROS handles several common use cases of robotics software development.

II. DESIGN GOALS

We do not claim that ROS is the best framework for all robotics software. In fact, we do not believe that such a framework exists; the field of robotics is far too broad for a single solution. ROS was designed to meet a specific set of challenges encountered when developing large-scale service robots as part of the STAIR project [2] at Stanford University[1] and the Personal Robots Program [3] at Willow Garage,[2] but the resulting architecture is far more general than the service-robot and mobile-manipulation domains.

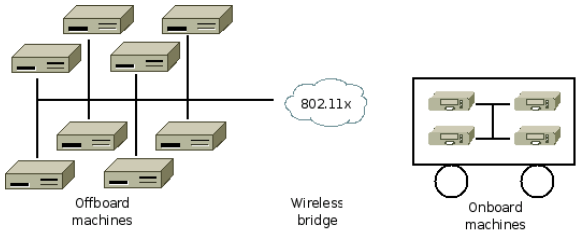

Fig. 1. A typical ROS network configuration

The philosophical goals of ROS can be summarized as:

- Peer-to-peer
- Tools-based
- Multi-lingual
- Thin
- Free and Open-Source

To our knowledge, no existing framework has this same set of design criteria. In this section, we will elaborate these philosophies and shows how they influenced the design and implementation of ROS.

A. Peer-to-Peer

A system built using ROS consists of a number of processes, potentially on a number of different hosts, connected at runtime in a peer-to-peer topology. Although frameworks based on a central server (e.g., CARMEN [4]) can also realize the benefits of the multi-process and multi-host design, a central data server is problematic if the computers are connected in a heterogenous network.

For example, on the large service robots for which ROS was designed, there are typically several onboard computers connected via ethernet. This network segment is bridged via wireless LAN to high-power offboard machines that are running computation-intensive tasks such as computer vision or speech recognition (Figure 1). Running the central server either onboard or offboard would result in unnecessary

[1] `http://stair.stanford.edu`
[2] `http://pr.willowgarage.com`